Planes preventivos en el mapa — cómo funciona

Nov 9, 2026 · @cristian

Resumen

Planes Preventivos pinta en el mapa hasta 8 puntos por cuadrante (PL1, PL2…) con hora, los 2 delitos más probables y el plan sugerido (p. ej. "Baliza", sitio, actividades, responsable). No se calcula en vivo: el backend Java solo lee GeoJSON ya generados en S3 (`pon-prod-ml-iapol/recomendaciones/`) por un pipeline de AWS Glue que corre cada semana y reentrena cada mes.

- **Toggle "Planes Preventivos"** (panel de capas): muestra u oculta los puntos.
- **Botón "¡Información planes preventivos!"**: abre un panel con los cuadrantes de la unidad que **no** tienen plan por falta de datos de entrenamiento.

Flujo en el front

La capa se monta sola en cuanto hay unidad y turno seleccionados; el toggle solo cambia su visibilidad.

1. **Montaje** — `MapPage.js` renderiza `PlanesRecomendacionesLayer` si `unitSelected && turnoSelected.id_turno` . Recibe `id_unidad` , `sigla_dmce` , el `turno` , la hora del turno y los cuadrantes fusionados.
2. **Fetch** — el componente llama `POST /optimizacion/geojson_retorno {id_turno, codigo: id_unidad, cad: sigla_dmce}` cada vez que cambian unidad, turno u hora.
3. **Limpieza** — si la respuesta llega como string, reemplaza `NaN` por `null` y la parsea. Si hay una hora puntual (no un rango del turno), filtra los puntos cuya `hora_inicio` coincida con esa hora.
4. **Delitos** — ordena `SIEDCO` y `RNMC` por `Probabilidad` descendente y los renombra a `Delitos` y `C3` (lo que lee el popup).
5. **Orden PL1, PL2…** — `assignOrderWithConditionalGrouping` numera los puntos por cuadrante según `hora_fin` . Si la fusión de demanda está lista, agrupa por cuadrante fusionado (campo `antiguo`) y deja solo los **8 de mayor probabilidad SIEDCO** por grupo.
6. **Pintado** — `setData` sobre la fuente `planesRecomendacionesData` : un `circle` (`planesRecomendacionesLayer-background`) y un `symbol` con la etiqueta `orden` . El clic lo maneja el handler global de `MapPage` con `PlanesRecomendacionesPopUp` .

7. **Estado padre** — `updatePlanesRecomendaciones` guarda `planesRecomendacionesPoints` y `servicioConErrorPlanes`.

El toggle (`ToggleOptions.js` , "Planes Preventivos") solo aparece si existe `planesRecomendacionesPoints` y no hay una predicción de demanda seleccionada (`showTogglePlanes`). Al activarlo pone `togglePlanesRecomendaciones = true` (capa visible) y abre el panel `showMensajePlanes` .

El botón "¡Información planes preventivos!" (icono `fa-land-mine-on`) abre `PlanesMensaje` , que llama `POST /optimizacion/info_cuadrantes_sin_data {sigla, unidad}` y lista, por unidad, los cuadrantes **sin** plan en RNMC y SIEDCO. Si la capa vino vacía o con error muestra "No hay planes preventivos para esta unidad"; si ambas listas vienen vacías, sugiere revisar las unidades hijas.

API consumida

Ambos endpoints están en `pon-xcrime-appServices` (`MainREST.java` → `CuadrantesServicio.java`) y solo leen de S3; no hay base de datos ni cálculo en la petición.

Endpoint (POST)	Body	Qué lee en S3 (`pon-prod-ml-iapol`)	Respuesta
`/optimizacion/geojson_retorno`	`id_turno` (`yyyymmdd_N`), `codigo` , `cad`	`recomendaciones/analytics/<año>/<lunes de la semana>/<cad>/<unidad>/prediccion_delitos_<cad>_<unidad>_<id_turno>.geojson` para la unidad **y todas sus hijas** (recursivo)	Un `FeatureCollection` con todos los puntos concatenados; vacío si ningún archivo existe
`/optimizacion/info_cuadrantes_sin_data`	`sigla` , `unidad`	`recomendaciones/artifacts/<sigla>/<unidad>/rango_<1.º del mes hace 1 año>_<fin del mes pasado>/{RNMC,SIEDCO}/cuadrantes_sin_data.json` (hoy `rango_20251101_20261031`)	`{RNMC: {<nombre unidad>: [cuadrantes]}, SIEDCO: {…}}` ; si la unidad tiene hijas, une las de cada hija

Cada punto del GeoJSON trae: `longitud` , `latitud` , `hora_inicio` y `hora_fin` (ventana de 40 min, formato `2026-11-10 01:00 PM`), `cuadrante` y, por fuente (`SIEDCO` , `RNMC`), los 2 delitos con `Probabilidad` , `plan` , `sitio` , `actividad` y `responsable` .

Ejemplo verificado: MEBOG / 13645, turno `20261110_2` → 7 puntos en el cuadrante `MEBOGMNVCCC04E03C07000019` ; el primero sugiere "Baliza" en "Vía Pública" para hurto a personas (probabilidad 0,32).

Backend y AWS

Los datos los produce el stack CDK `Cdk-Planes-Recomendaciones` (cuenta prod 926162397524, us-east-1): jobs de AWS Glue (Python shell) disparados por reglas de EventBridge. Últimas corridas exitosas: entrenamiento 2026-11-01, puntos 2026-11-02, consumo 2026-11-03.



pipeline de planes preventivos · 6 jobs Glue + backend

Los modelos se reentrenan una vez al mes; cada semana se reutilizan para generar los puntos de la semana siguiente.

Cómo se elige cada punto (`opt_points`): toma la predicción de calor del turno, cruza cada cuadrante con sus celdas y descarta el 20 % de menor intensidad × área. Sobre una rejilla de 30 × 30 puntúa cada punto por cercanía (suma de 1/distancia) a esas celdas. Se queda con los 8 mejores y los mueve a la calle más cercana.

Cómo se arma la recomendación (`consume_models`):

- **Hora**: la hora con más ocurrencias históricas del cuadrante dentro del turno (`lookup_hours`, una por ranking de punto); la ventana dura 40 min.
- **Delitos**: el RandomForest del cuadrante devuelve la probabilidad de cada delito; se guardan los 2 más altos, por SIEDCO y por RNMC.
- **Sitio**: la fuente externa (colegio, comercio, etc.) que contiene el punto o está a menos de `max_distance`; si no hay, "Vía Pública".

- **Plan, actividad y responsable**: se busca `delito-sitio` en la homologación de la unidad y en `lookup-files/matriz-validacion-planes-DEFINITIVA.csv`.

Cuadrantes sin datos: `train_models` no puede entrenar un cuadrante sin historia; lo anota en `cuadrantes_sin_data.json`. Ese archivo es el que lee el botón de información.

Observaciones y riesgos

1. **El turno 1 del domingo se pierde.** El backend busca en la carpeta del lunes de la semana del `id_turno`. `20261108_1` (domingo) se busca en `20261102/`, pero el pipeline lo escribió en `20261109/` (verificado en S3 para MEBOG/13645). Resultado: mapa vacío para ese turno.
2. **Hay azar en el plan y el sitio.** `random.choice` elige la fuente externa, el sitio y, si no hay coincidencia, el plan. Volver a correr el job da recomendaciones distintas para el mismo punto.
3. **Error que se ve como "tiene planes".** En unidades padre, si una hija no tiene `cuadrantes_sin_data.json` el Java hace `RNMC.isEmpty()` sobre `null` (posible NullPointerException → 500). `PlanesMensaje` guarda el error pero no lo muestra, y termina diciendo "verifique las unidades hijas". Visto por código, no reproducido.
4. **Dependencia de la predicción de calor.** Si la predicción semanal de la unidad sale en cero (caso DESAB) o falta, no hay puntos óptimos y la capa sale vacía.
5. **Etiquetas PL sin fusión.** Si la Fusión de Demanda no está lista, el orden PL se calcula por cuadrante original sin el tope de 8 por grupo fusionado.
6. **Tildes rotas.** El ejemplo de S3 trae `art�culo`, `V�a P�blica`: el archivo no se lee o escribe en UTF-8 en algún punto. Pendiente confirmar si se ve así en el popup.
7. **Planes Preventivos ≠ Planes GESCOV.** Esta capa es una recomendación del modelo, no un plan programado o en ejecución.